

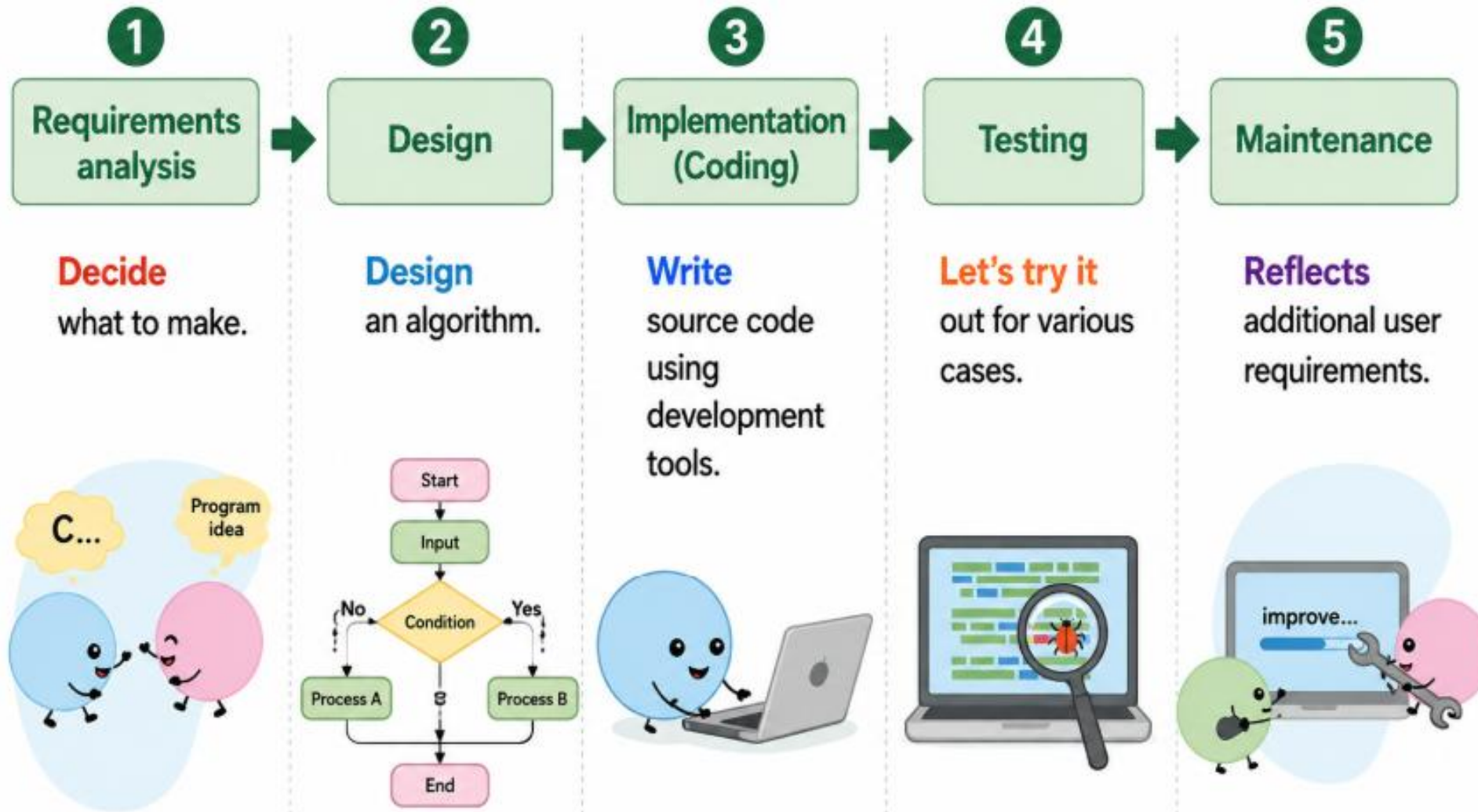
C Programming

Program Writing Process & C Program Components

Ch. 2

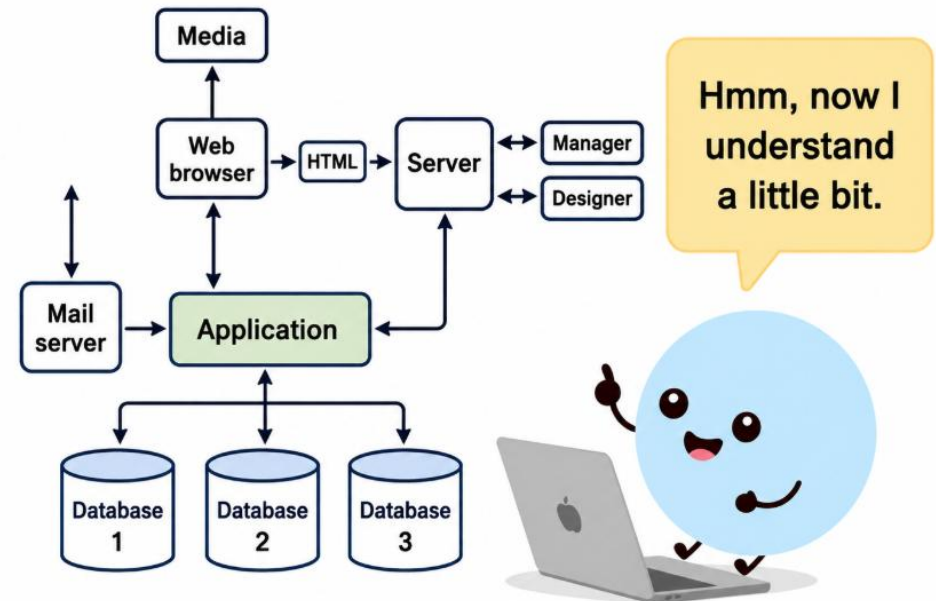
Program Development Process

Program Development process



Design

- Steps to develop an algorithm to solve a problem.
- Using flowcharts and pseudocode as tools.
- Algorithms are independent of programming language.
- An algorithm focuses on the steps that must be taken to achieve a desired result.



Write Source Code

- Describe each step of the algorithm using a programming language.
- An algorithm written in the grammar of a programming language is called a source program.
- Source programs are usually written using a text editor or integrated development environment.
- Source file name: (example) test.c

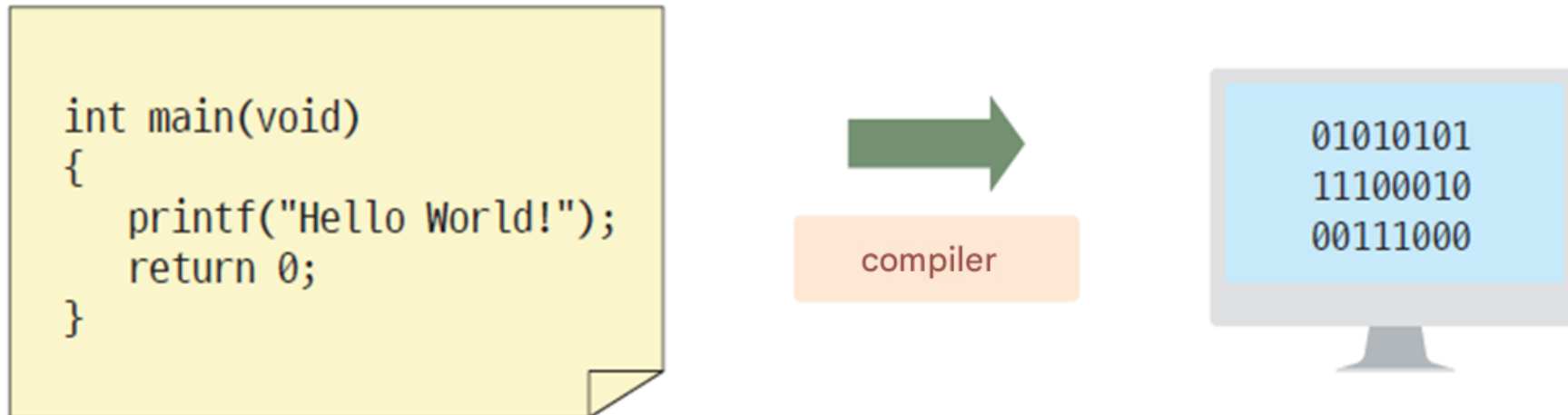


editor

```
int main(void)
{
    printf("Hello World!");
    return 0;
}
```

Compile

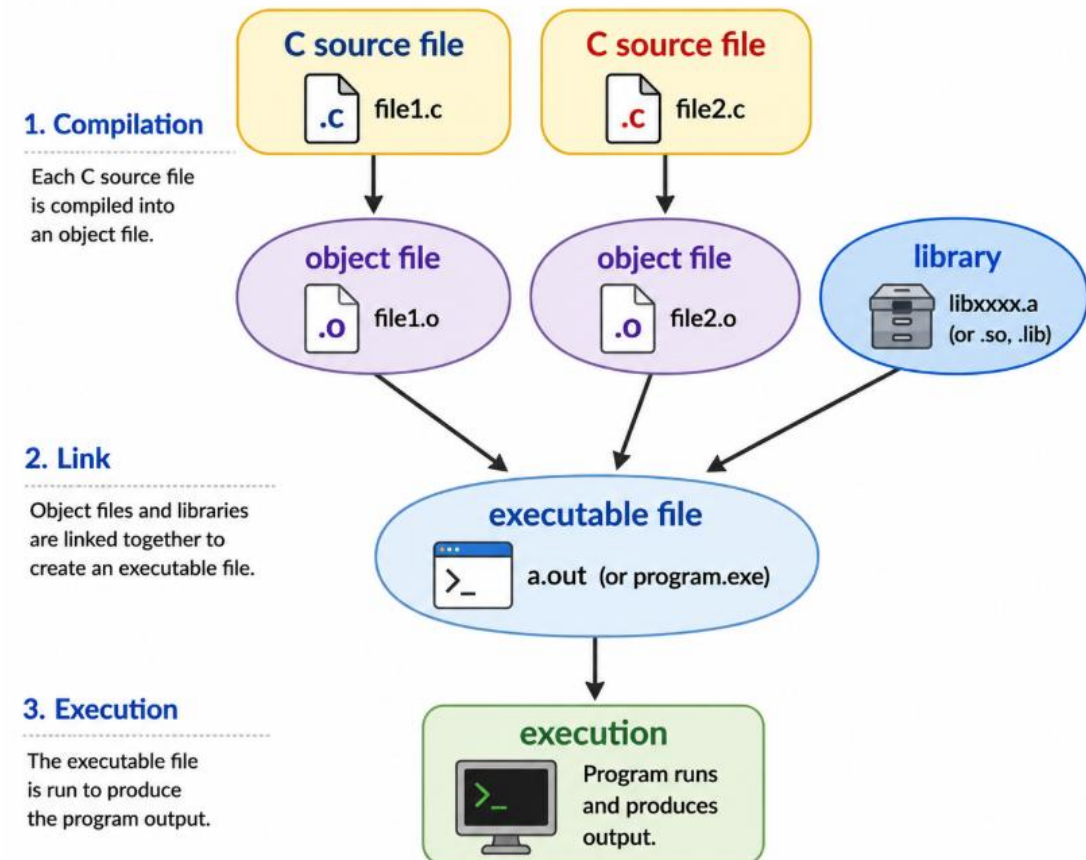
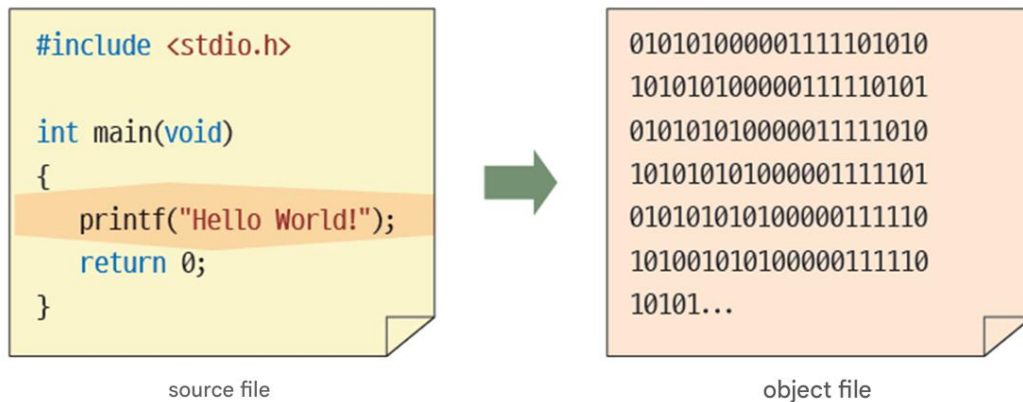
- The task of converting a source program into an object file.
- Object file name: (example) test.obj / test.o
- Compile error: a syntax error - for example, "He go to school;" is grammatically wrong.



Link

- Executable file name: (example) test.exe / test
- Library: pre-written functions that programmers often use (example: input/output functions, file processing, mathematical function calculations).
- The program that performs linking is called a **linker**.

- Object file

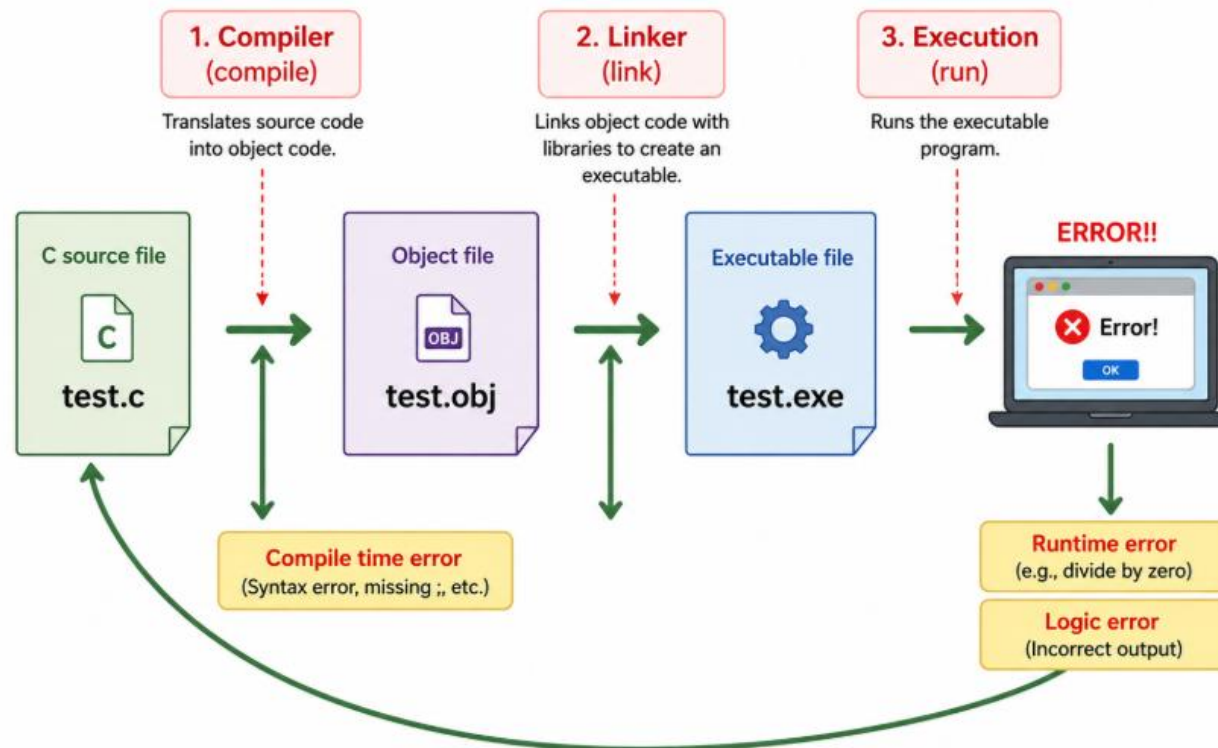


Running and Debugging

- Run-time error: e.g. division by zero, accessing an invalid memory address.
- Logical error: grammatically correct, but logically incorrect.

Example: ① Prepare bowl 1 and bowl 2. ② Add flour, milk, and eggs to bowl 1 and mix well. ③ Place bowl **2** in the oven - bake at 350 degrees for 30 minutes. (The logic is wrong: **nothing was ever put in bowl 2!**)

- Debugging: catching errors that exist in the **source code**.



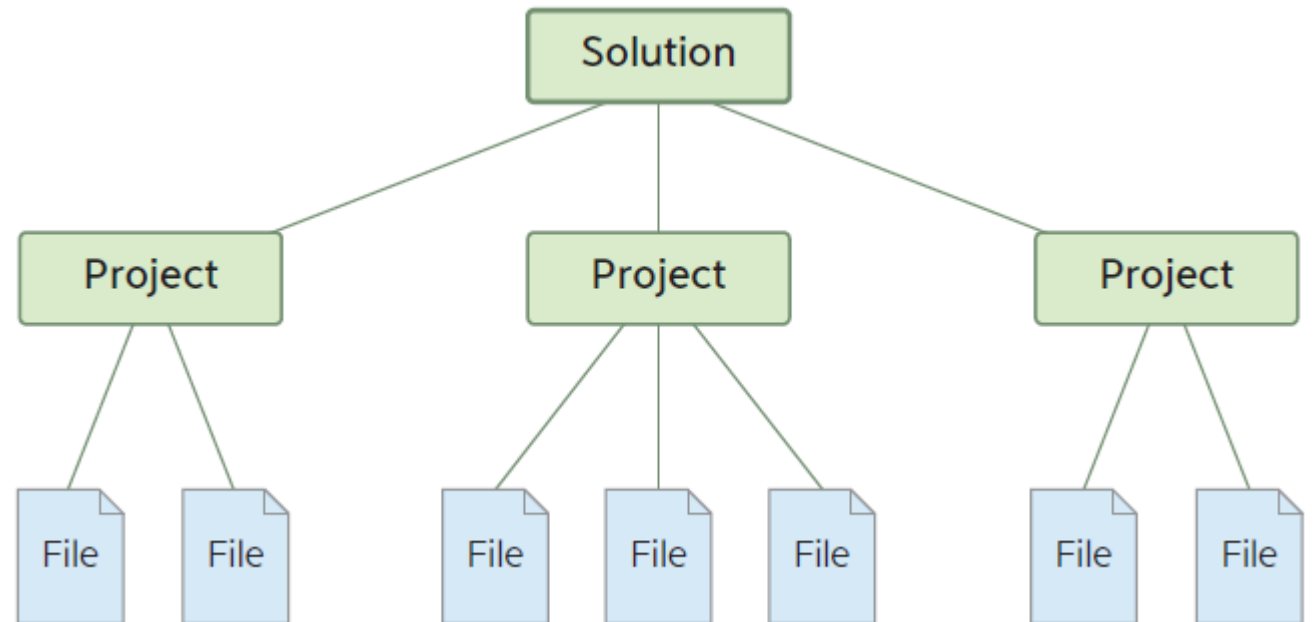
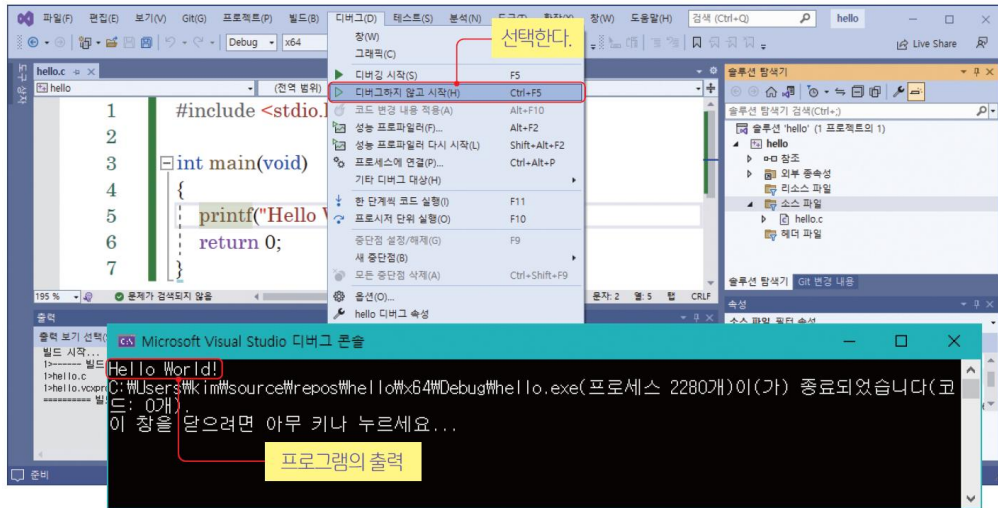
Debugging

- Catching errors that exist in the source code.



Integrated Development Environment (IDE)

- Integrated development environment (IDE) = editor + **compiler** + debugger.
- Examples: Visual Studio (Microsoft), Eclipse (open source), Dev-C++ (open source).
- Solution: a container containing the projects needed to be resolved.
- Project: a container that contains several items needed to create a single executable file.



Description of the First Program

Practice

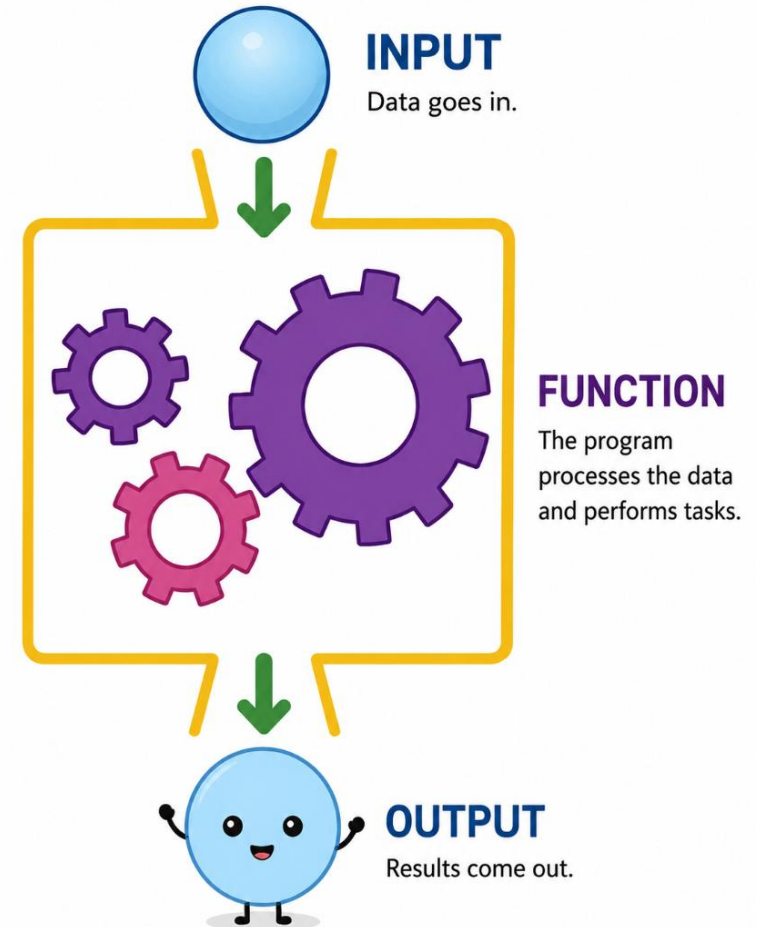
```
#include <stdio.h>

int main (void)
{
    printf( "Hello World!" ) ;
    return 0;
}
```

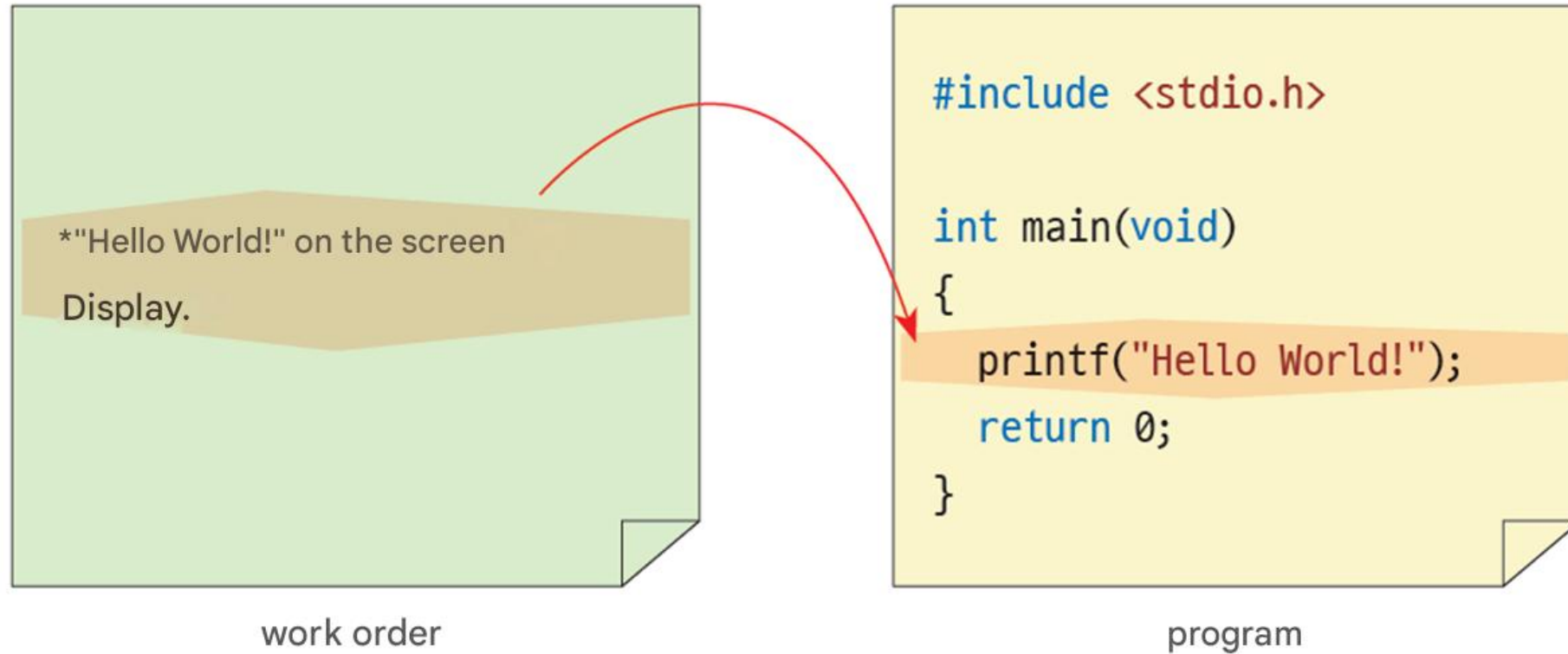
hello.c — Program == Work Instructions: a program is a document that tells the computer what to do.

Function

- Function: a standalone piece of code written to perform a specific task.
- A program = a set of functions.
- Sentence (statement):
a function consists of several statements.
- Statements are executed sequentially,
and there must be a ; at the end of a statement.



Program == Work Instructions



Where to write down your work

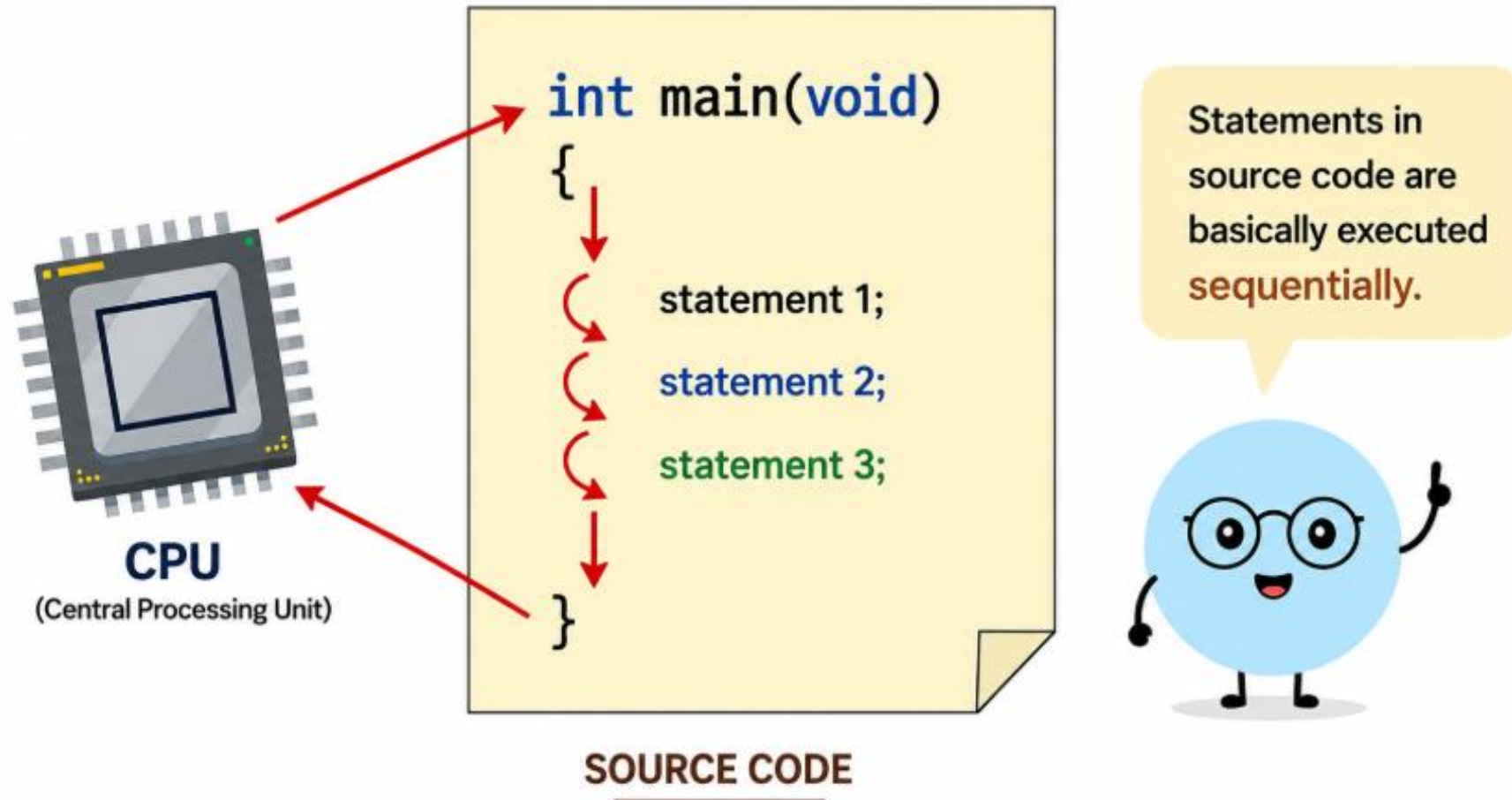
```
#include <stdio.h>

int main(void)
{
    
    return 0;
}
```

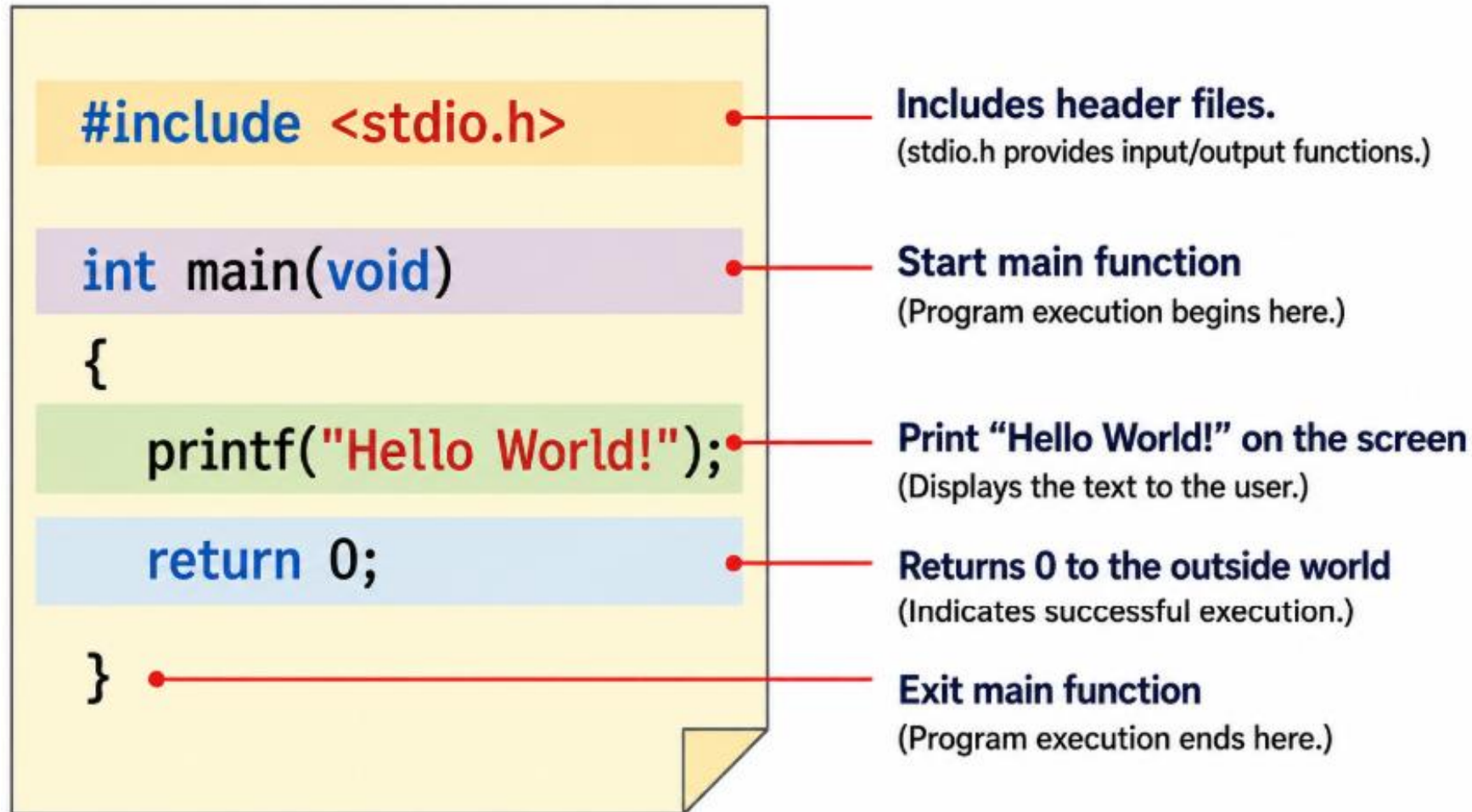
program

Write the sentence that
performs the desired task
here.

Brief source description

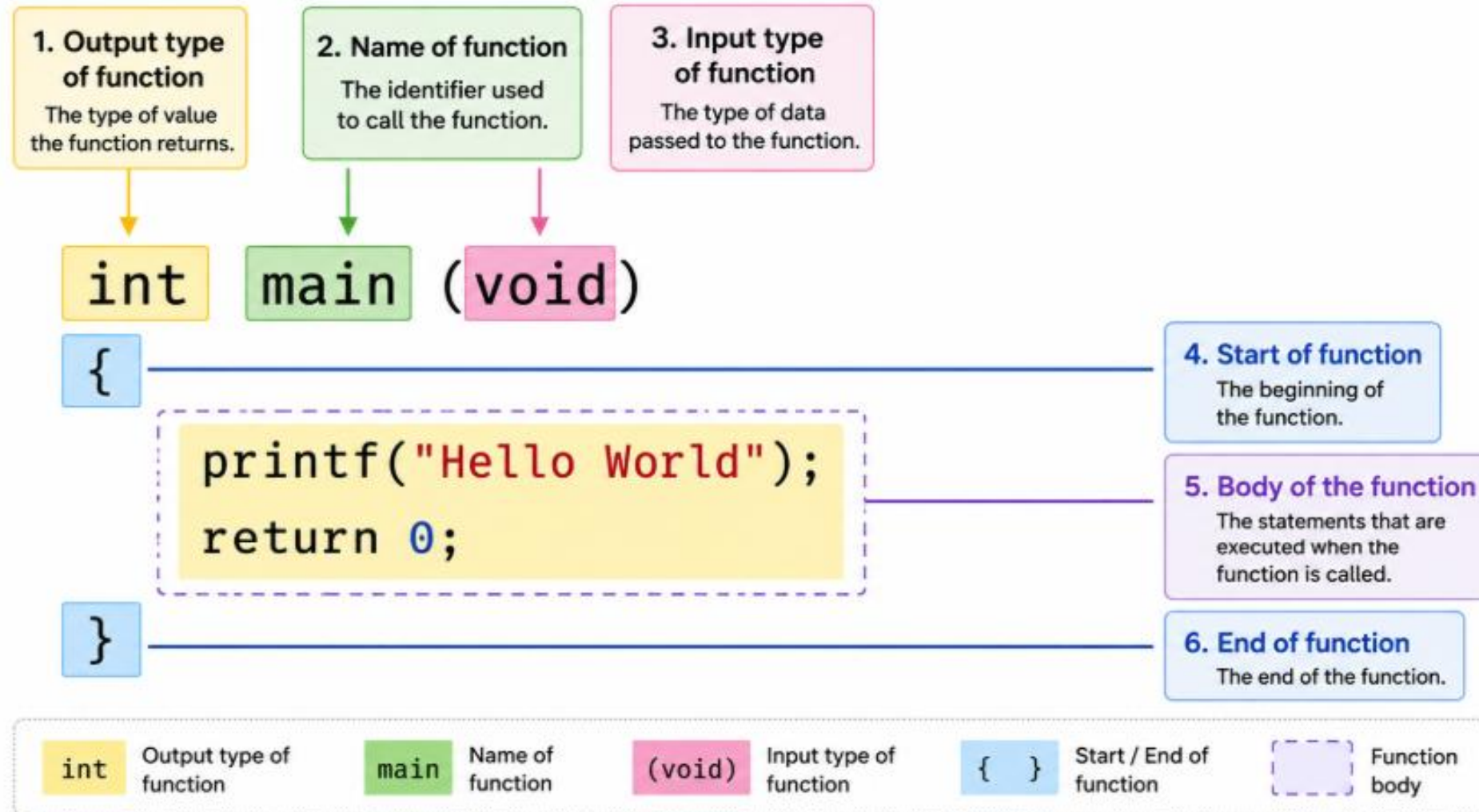


Brief source description



Program

Brief source description



Brief source description

1 #include

Includes a specific file in the current location within the source code.

2 Caution! Preprocessor directive

Statements must not end with a semicolon (;).

#include <stdio.h>

3 Header file: The compiler
A file containing the information
you need.

- **stdio.h:** standard input output header file

What Is a Library in C?

Think of making a pizza.

PIZZA LIBRARY

Ready-made reusable ingredients



Dough function



Cheese function



Topping function

C PROGRAM

```
#include <pizza.h>
```

```
make_dough();  
add_cheese();  
add_toppings();
```

Goal

Make different kinds of pizza

A library is a toolbox of reusable code that has already been prepared for you.

Brief source description

1 **printf()** is a function provided by the compiler that handles output.

printf("Hello World!");

2 The string within the double quotes is printed on the screen.



Brief source description

1 return of the function

Return the result to the outside world.

return 0;

2 Return value is 0

The function returns the value 0 to the outside world.

Error Fixing and Debugging

- Errors may occur during compilation or during execution.
- Error vs. Warning: error = fatal error; warning = minor error.
- Compile-time errors: mostly grammatical errors.
- Runtime errors: errors such as division by 0 (zero).
- Logical error: an error that is logically incorrect and results in an outcome that is not what was intended.

```
1  #include <stdio.h>
2
3  int Main(void)
4  {
5  !  int x;           // uninitialized variable
6  !  printf("Hello, %s", name); // name is undefined
7  x  if (x = 5) {    // assignment instead of comparison
8      printf("x is 5\n");
9  }
10 !  printf("This line has no newline");
11 x  return 0
12 }
```

Header file inclusion (info)
Includes the standard input-output library.

Warning: unused variable
Variable 'x' is declared but not initialized.

Warning: implicit declaration
Function 'name' is not declared.

Error: assignment in condition
Did you mean '=='?

Warning: missing newline
Format string does not end with \n.

Error: missing semicolon
Expected ';' before '}'.

Program Output:

```
main.c:5:5: warning: unused variable 'x' [-Wunused-variable]
main.c:6:12: warning: implicit declaration of function 'name'
main.c:7:9: error: assignment makes integer from pointer without a cast
main.c:10:41: warning: missing newline at end of format string
main.c:11:12: error: expected ';' before '}' token
Compilation failed.
```

Ch. 3

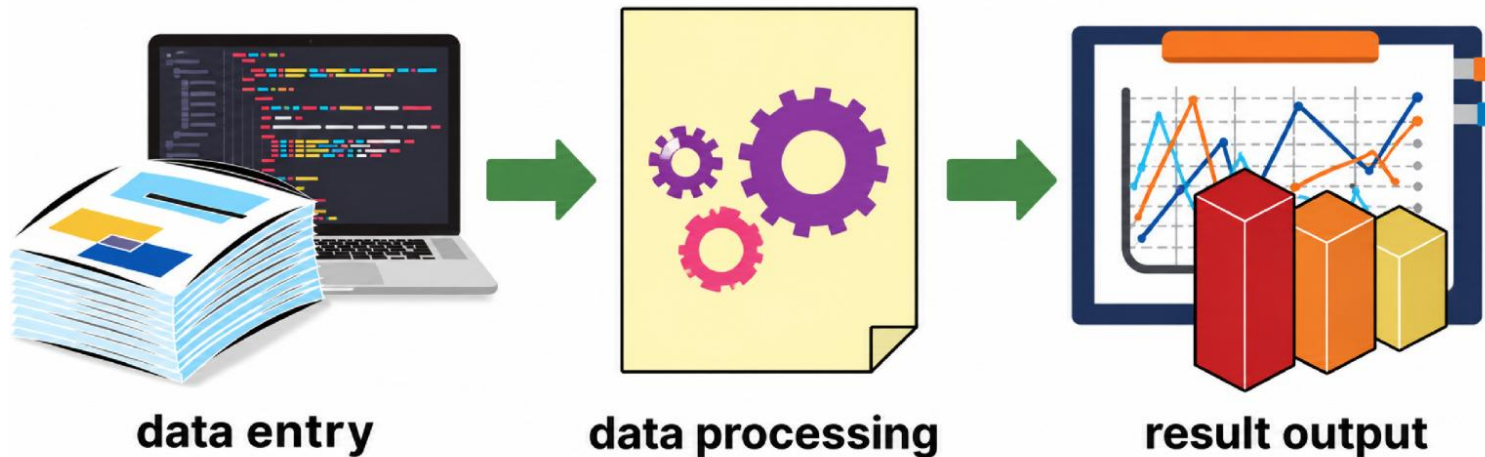
C Program Components

What You Will Learn in This Chapter

- Annotation (comments), variables, constants
- Function, sentence
- Output function `printf()`, input function `scanf()`
- Arithmetic operations, assignment operation
- In this chapter, we will look at the components that make up a C program.

General Program Form

- Most programs follow the same general form:
 - receiving data (input stage)
 - processing the data (processing stage)
 - displaying the results on the screen (output stage)



Without Comments

```
#include <stdio.h>
int main( void )
{
    int x;
    int y;
    int sum;

    x = 100;
    y = 200;
    sum = x + y;
    printf( "Sum of two numbers: %d" , sum);
    return 0;
}
```

With Comments

```
/* Program to calculate the sum of two numbers */
#include <stdio.h>
int main( void )
{
    int x; // variable to store the first integer
    int y; // variable to store the second integer
    int sum; // Variable that stores the sum of two integers

    x = 100;
    y = 200;
    sum = x + y;
    printf( "Sum of two numbers: %d" , sum);
    return 0;
}
```

Comments are text that explains the code.

Two Ways to Comment

- `/* single line comment */`
- `/* Several lines`

File name : add.c

Description : Program to add two numbers

Author : Hong Gil-dong

`* /`

- `// This entire line is a comment.`
- `int x; // From here to the end of the line is a comment.`

- The importance of comments: when someone else looks at the program, it is much easier to understand the contents if there are comments.

A good comment does not repeat the code - it should clearly state the intent of the code.

Indentation

- **Indentation**: indenting statements on the same level a few characters from the left end.
- Without comments and indentation, a program still runs, but it is difficult to know what kind of processing the program is doing, and it is also difficult to distinguish statements on the same level.

```
#include <stdio.h>

int main(void)
{
    int x;
    int y;
    int sum;
    ...
    return 0;
}
```

Insert blank lines to differentiate between meanings.

Explain the intent of the program in comments.

// Variable to store the first integer
// Variable to store the second integer
// Variable that stores the sum of two integers

If the content is the same, indentation is applied.

[Bad case 1]

```
#include <stdio.h>

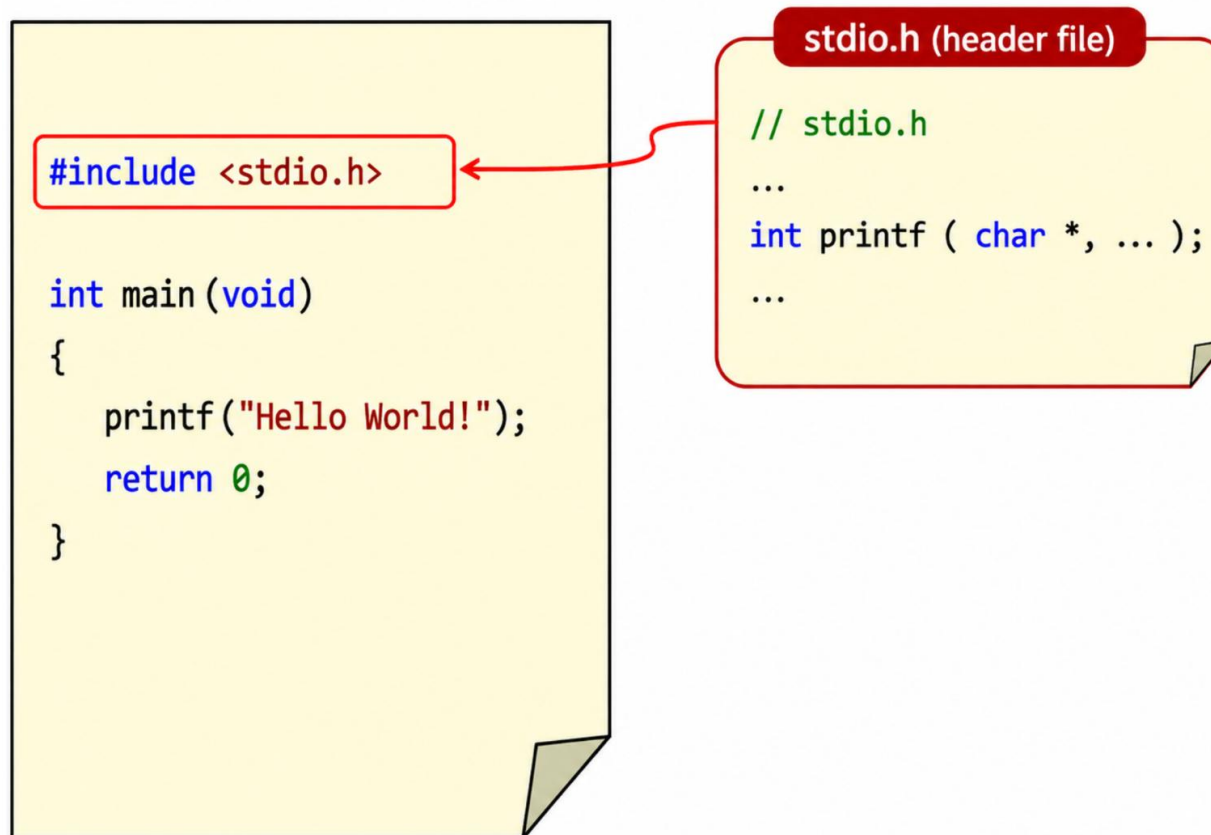
int main(void)
{
    int x; // Variable to store the first integer
    int y; // Variable to store the second integer
    int sum; // Variable that stores the sum of two integers
    ...
    return 0;
}
```

[Bad case 2]

```
#include <stdio.h> int main(void){int x;int y;int sum;...return 0;}
```

The Preprocessor

- `#include <stdio.h>`: a preprocessor directive - it starts with a `#` symbol.
- `stdio.h` contains definitions of library functions for standard input and output.



Function (Recap)

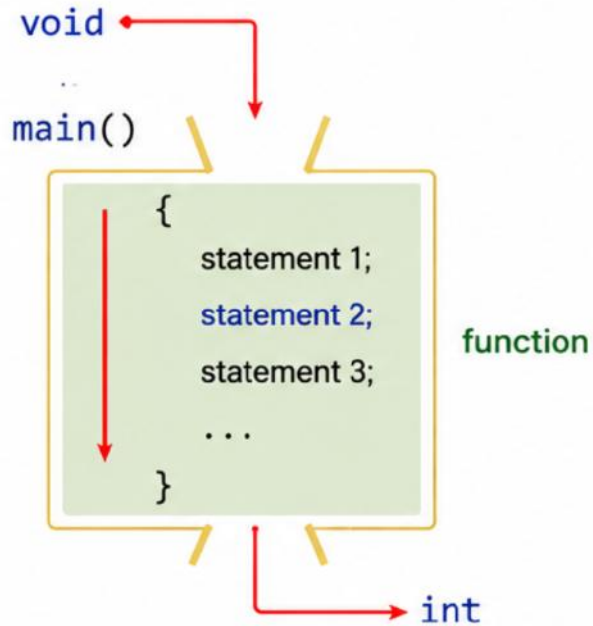
- Function: a set of processing steps that perform a specific function, grouped in braces { } and named accordingly.
- Functions are the basic units that make up a program.
- Inside a function, the processing steps (statements) that the function processes are listed in curly brackets.
- return: a **keyword** that returns a value while terminating a function. To return a value, write the return value after return.

Function (Recap)



A **function** is a block of code that performs a specific task.
The **return** statement sends a value back to the caller (usually **0** in main).

1. What is a function?



`void` → no return value
`int` → returns an integer value

2. Anatomy of a simple function

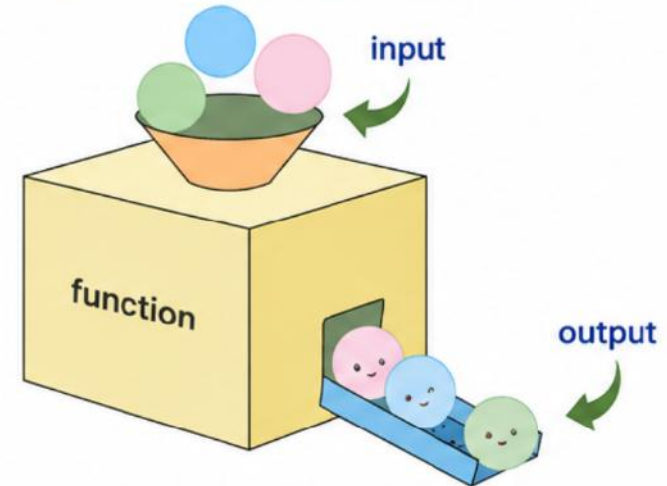
```
#include <stdio.h>

int main(void) {
    ...
    ...
    return 0;
}
```

return statement

- Ends the function execution.
- Sends a value back to the caller.
- In main, `return 0;` means the program ended successfully.

3. How a function works



4. Example Flow (main function)

- 1 Program starts at `main()`
- 2 Statements are executed inside `{ ... }`
- 3 `return 0;` is reached
- 4 Control returns to the operating system
- 5 Program ends (`0` = success)



Summary

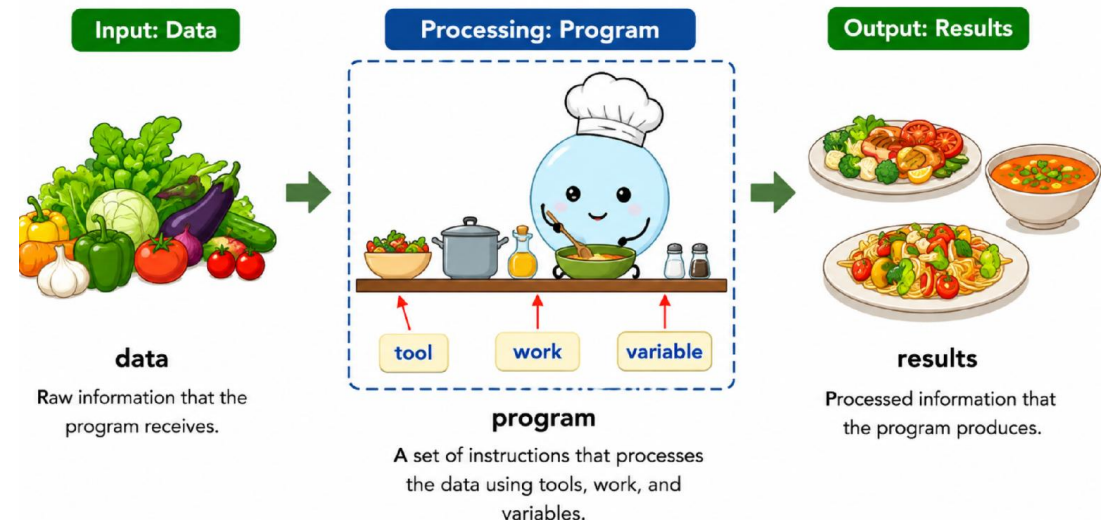
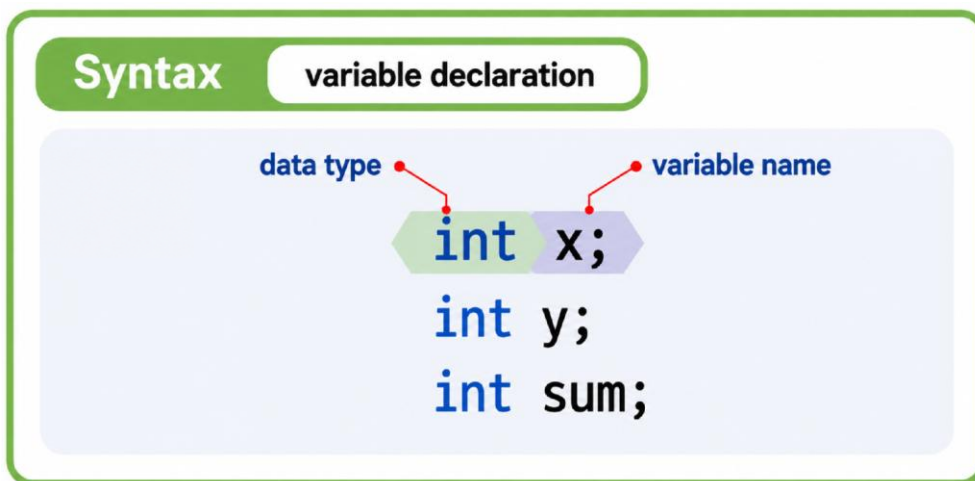
- A function groups code to do a task.

`void` → no value returned

`int` → returns a value (e.g., `0`)

Variable

- **Variable**: memory space used for temporarily storing data used by a program.
- Why are variables needed? They serve to temporarily store data values - for example, data received from the user.
- A **variable** can be thought of as a box that holds data: `int x;` (**data type**) (the name of the **variable**)
- **Data type**: specifies whether the data to be stored in the **variable** is an integer, a real number, or some other data.



Declaring Variables

```
int x ; // first Integer Save Variable  
int y ; // second Integer Save Variable  
int sum; // two Integrity Sum Save Variable  
  
// Declaring multiple variables on one line - also possible!  
int x, y, sum;
```

Variable declaration: telling the compiler in advance what type of variable will be used.

Name of Variable: Identifier Rules !!

- Identifiers consist of **English letters, numbers, and the underscore character _**.
- There must be no spaces in the middle of the **identifier**.
- The first character must be a letter or underscore _ ; an **identifier** cannot start with a number.
- Uppercase and lowercase letters are distinguished - index, Index, and INDEX are all different variables.
- Identifiers that are identical to **keywords** in the C language are **not allowed**.

[Keywords in C]

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Good Variable Names

- You should choose a name that best describes the role of the **variable**.
- i, j, k (X) vs. year, month, date (O)
- How to create a multi-word name: underscore method (bank_account), or capitalize the first letter of each word (BankAccount).

Initialization of Variables

```
int x = 10;  
int y = 20;  
int sum = 0;
```

```
// If the variables are of the same type, you can declare and  
// initialize them on the same line.
```

```
int width = 100, height = 200;
```

```
// This is not syntactically incorrect, but should be avoided:
```

```
int width, height = 200;    // width is not initialized!
```

Storing Values: The Assignment Operation (How to store)

- Formula (expression): operands and operators consisting of an expression. A formula has a result.
- Assignment operation: an operation that stores a value in a **variable**, using the = symbol.
- `x = 100;` (100 is stored into x)
- A **variable** can also be assigned the value of another **variable**, and its value can be changed as many times as you like.

Arithmetic operations with variables !!

- Arithmetic operators are similar to the operation symbols commonly used in mathematics.

Calculation	Operator (in C)	C Formula (Example with x and y)	Symbols in Mathematics
 Addition	+	$x + y$	$x + y$
 Subtraction	-	$x - y$	$x - y$
 Multiplication	*	$x * y$	xy
 Division	/	x / y	x/y or $x \div y$
 Remainder (Modulus)	%	$x \% y$	$x \bmod y$

Arithmetic operations with variables

PROGRAM

```
#include <stdio.h>
int main(void)
{
    int x;
    int y;
    int sum;

    x = 100;
    y = 200;

    sum = x + y;

    printf("Sum of two numbers: %d", sum);

    return 0;
}
```

// 1. variable declaration

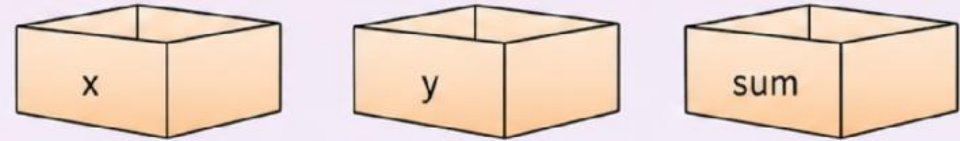
// 2. value assignment

// 3. addition operation

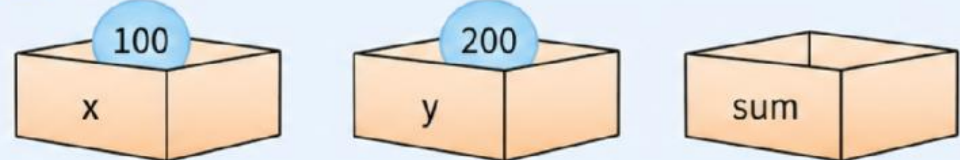
// 4. output

INSIDE THE COMPUTER

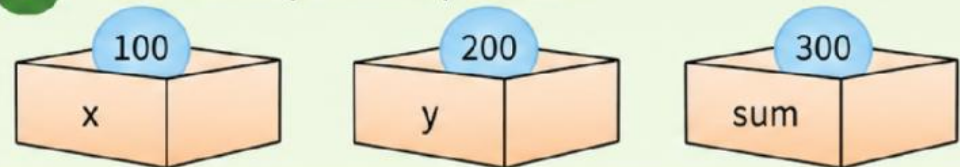
1 A variable is created.



2 A value is assigned to a variable.



3 An addition operation is performed.



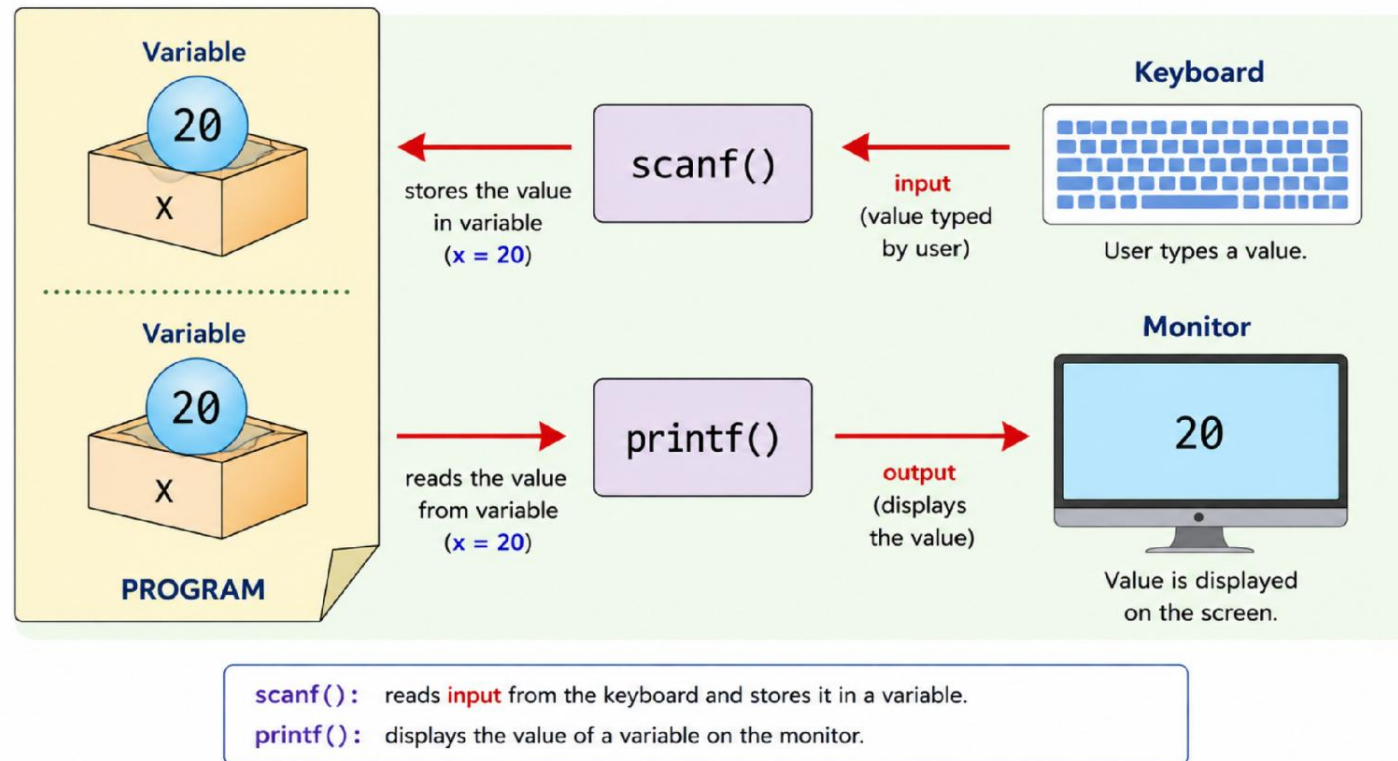
4 Output is produced.

Sum of two numbers: 300



Library Functions: printf() and scanf()

- Library functions: functions that the **compiler** provides for programmers to use.
- **printf()**: standard output function for printing to the monitor.
- **scanf()**: standard input function for input from the keyboard.
- String output: **printf("Hello World!\n");** - a string is a list of several characters together.



Output Format: Format Specifiers

```
int sum = 30;
printf ( "Sum of two numbers: %d\n" , sum);
// Output: Sum of two numbers: 30

// Format specifier: specifies the format in which values are
// printed in printf(). The value of the variable is substituted
// in the place of the format specifier, then printed.
// Caution! The data types of the format and variable must match.
```

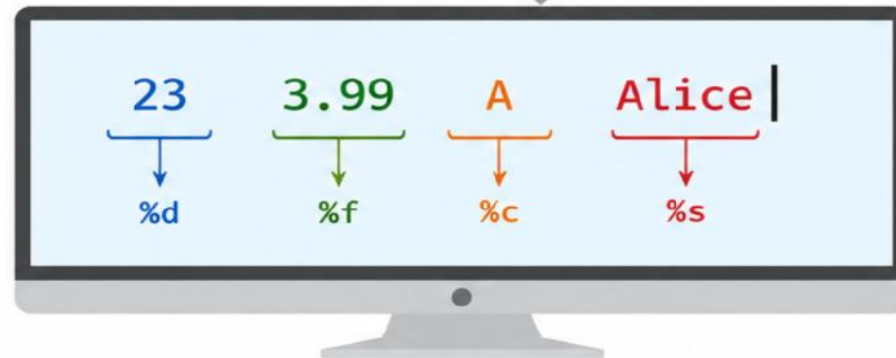
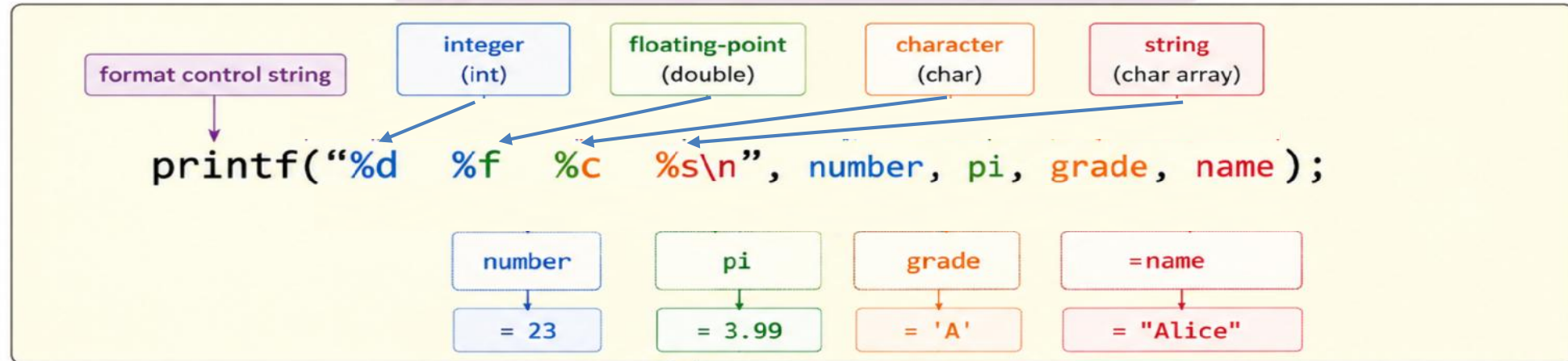
Format Specifiers for printf()

- Format specifier : Specifies the format in which values are printed in **printf()**.

Format Specifier	Meaning	Example	Execution Result
%d	Output as a decimal integer (whole number).	<code>printf("%d\n", 10);</code>	10
%f	Output as a floating-point number (decimal number).	<code>printf("%f\n", 3.14);</code>	3.14
%c	Output as a character.	<code>printf("%c\n", 'a');</code>	a
%s	Output as a string (sequence of characters).	<code>printf("%s\n", "Hello");</code>	Hello

Format Specifiers for printf()

printf() using all format specifiers



The number of format specifiers and the number of variables must be equal.



Specifier	%d	%f	%c	%s
Meaning	decimal integer	floating-point number	character	string (sequence of characters)
Example Output	23	3.99	A	Alice

scanf(): Getting a Value from the Keyboard

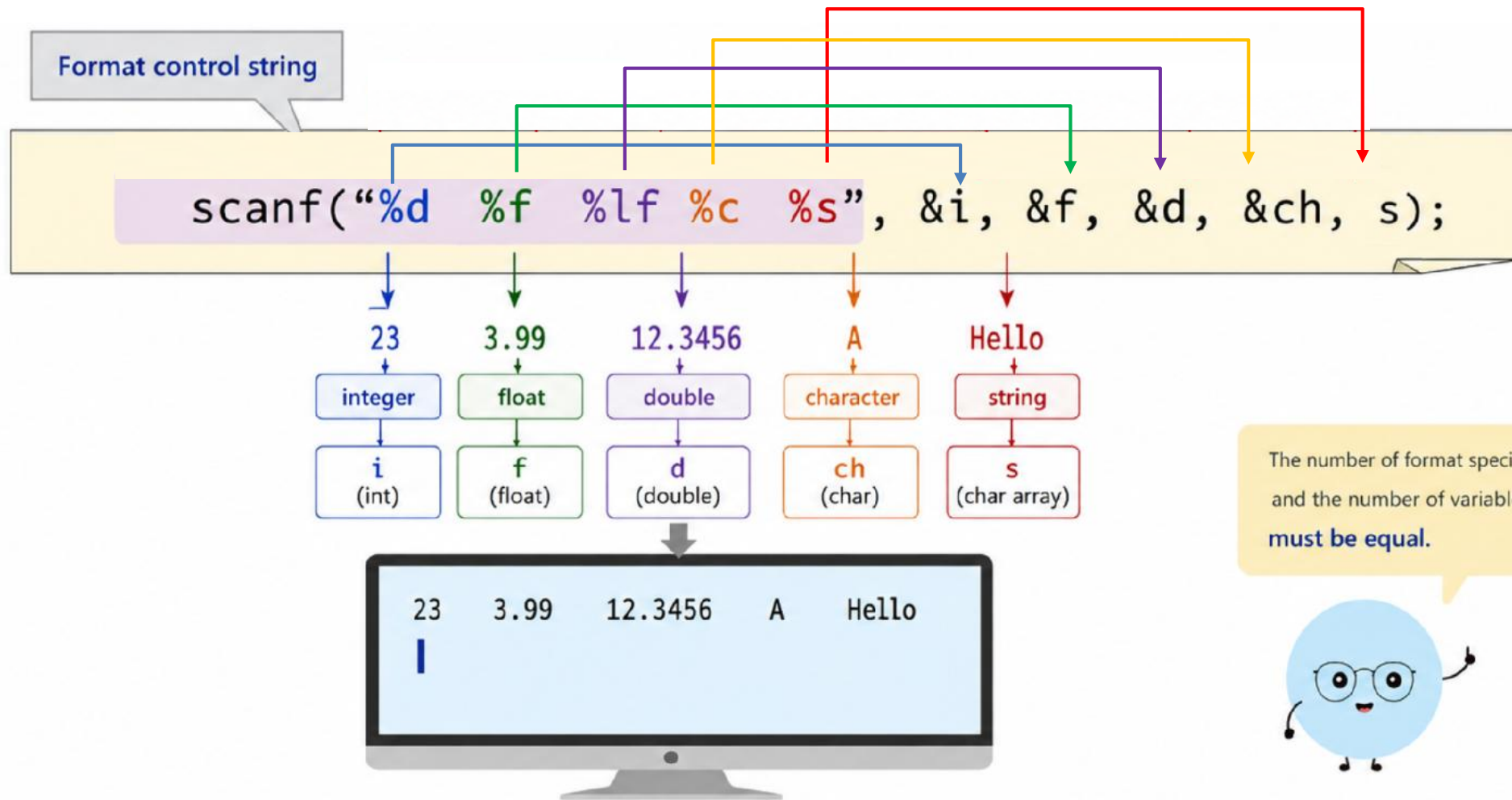
```
int x;  
scanf ("%d", &x);  
// Gets a value from the keyboard and stores it in variable x.  
// scanf() requires the ADDRESS of the variable.  
  
// Why do we need an address? Just like when we buy a product  
// online and have it delivered to our home, we have to tell  
// the shopping mall our address. The & operator calculates  
// the address of a variable.
```

Format Specifiers for scanf()

- Mostly similar to **printf()**.
- Gets a value from the keyboard and stores it in a variable.
- Requires the address of the variable

Format Specifier	Meaning	Example (scanf)
%d	Enter a decimal integer.	<code>scanf("%d", &i);</code>
%f	Enter a real number of float type.	<code>scanf("%f", &f);</code>
%lf	Enter a real number of type double.	<code>scanf("%lf", &d);</code>
%c	Enter one character.	<code>scanf("%c", &ch);</code>
%s	Enter a string.	<code>char s[10]; scanf("%s", s);</code>

Format Specifiers for scanf()



Specifier	%d	%f	%lf	%c	%s
Meaning	decimal integer	real number (float)	real number (double)	character	string
Example Input	23	3.99	12.3456	A	Hello

Lab 1: Four basic arithmetic calculation

Practice

- Write a C program that uses two integer variables.

- Set the values as follows:

```
int x = 20;
```

```
int y = 10;
```

- Calculate and print:

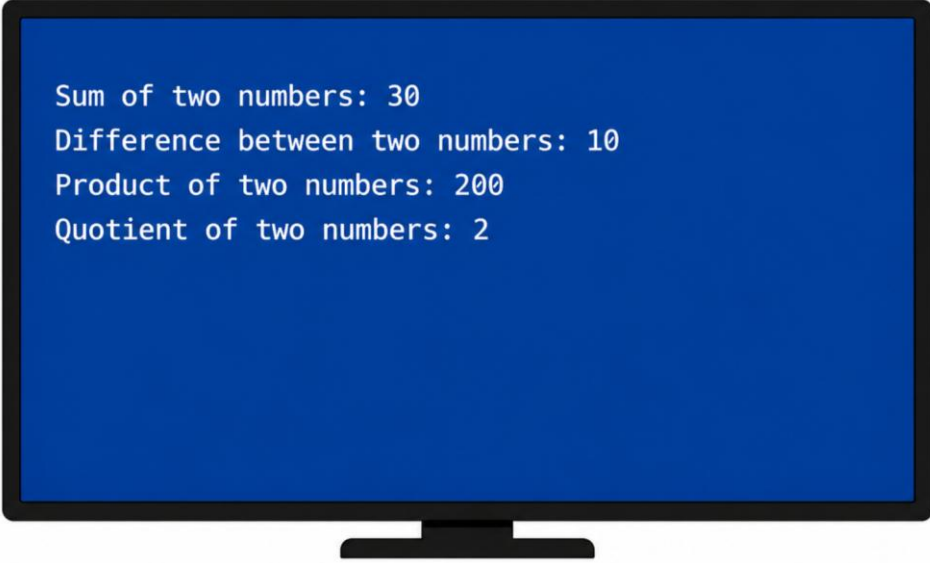
- 1.The sum of the two numbers.

- 2.The difference between the two numbers.

- 3.The product of the two numbers.

- 4.The quotient of the two numbers.

- Your program should produce the following output:



```
Sum of two numbers: 30  
Difference between two numbers: 10  
Product of two numbers: 200  
Quotient of two numbers: 2
```

Hint: Use the operators +, -, *, and /, and use printf() to display the results.

Solution?

Sum of two numbers	: 30
Difference between two numbers	: 10
Product of two numbers	: 200
Quotient of two numbers	: 2

For:

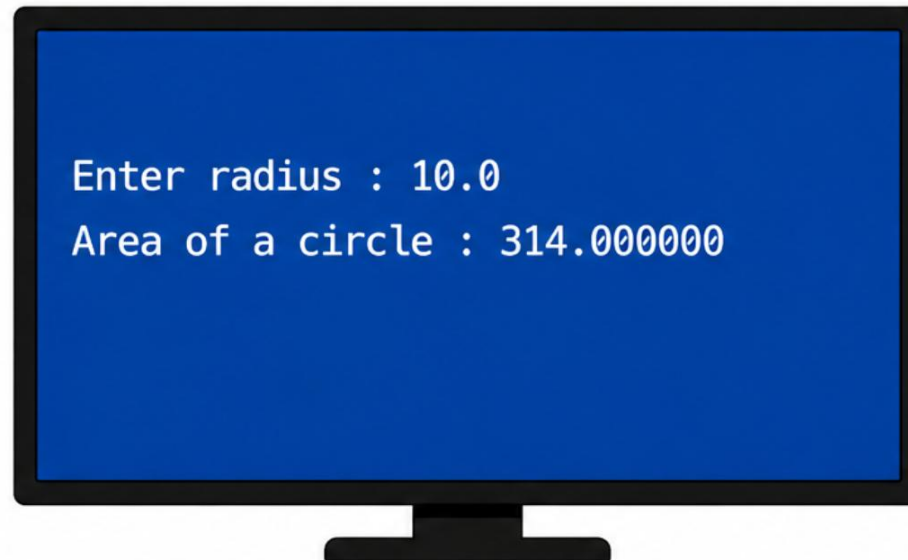
x = 20

y = 10

Lab 2: Circle Area Calculation Program

Practice

- receives the radius of a circle from the user , calculates the area of the circle , and then displays it on the screen.

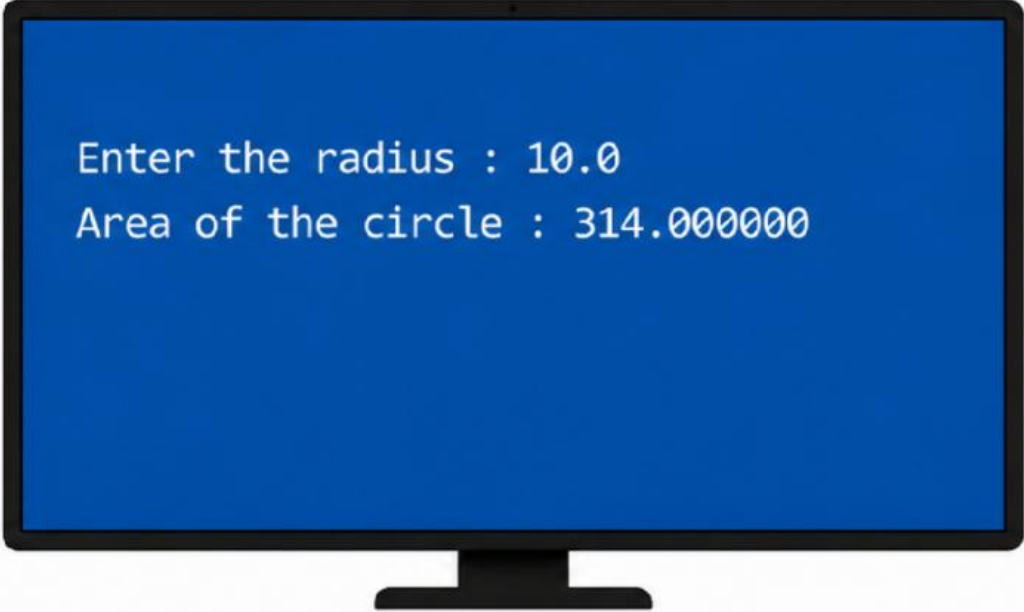


Hint: Use `scanf()` to receive radius.

Formula: $\text{Area} = 3.14 * \text{radius} * \text{radius}$

Solution?

RESULT ON THE SCREEN



```
Enter the radius : 10.0  
Area of the circle : 314.000000
```

Q & A

Lab & Quiz: right after this - see you in the practice session!